如何保证缓存和数据库的一致性

常见的缓存更新策略有几种?

旁路缓存

第一种就是cache aside旁路缓存. 我们平时业务中使用redis都是旁路缓存策略.

- 旁路缓存就是读请求先从缓存中读数据, 读不到就再从数据库中读, 然后写入缓存.
- 写的话就比较讲究了, 先写缓存还是先写数据库, 写缓存还是删除缓存, 怎么操作才能保证缓存和数据库的一致性, 这是个非常复杂的问题, 后面会详细聊.
- 旁路缓存这种方式的特点就是业务代码既要操作数据库又要操作缓存, 以数据库的数据为主, 缓存只是暂时存储数据而已.

read through/write through读穿写穿

第二种是read through/write through读穿写穿. 它的设计思想就是说, 不直接操作数据库, 只操作缓存, 然后让缓存自身去操作数据库.

- 读穿就是说, 每次读都读缓存, 如果缓存有, 就返回数据, 没有那缓存自己从数据库中读对应数据, 然后写缓存, 返回数据. 你只操作缓存, 至于从数据库读, 写缓存这些都由缓存中间件自身完成.
- 写穿就是说, 每次写数据, 就直接写缓存, 缓存自身去更新数据库.
- 也就是说, 我们只操作缓存, 读写数据库都交给缓存自身完成. 这就是读穿写穿.

write back写回

第三种是write back写回, 也有叫异步写缓存的, 但本质都是一样的.

- 写回和读穿写穿的思想差不多, 就是只操作缓存, 然后让缓存自身去操作数据库.
- 但是 不一样的点是, 写回是异步批量写数据库.
- 意思就是说, 写操作只写缓存, 只更新缓存, 然后让后台线程异步的批量的把缓存的更新写入数据库.
- 这种非常适合写多读少的场景, 但是因为是异步地写数据库, 所以如果突然宕机, 会有数据丢失的可能.

无论是读穿写穿, 还是写回, 目前做业务的缓存, 像redis或者memcached, 或者大厂自研的缓存中间件都没提供这种缓存自身和数据库交互的功能.

但是这种思想其实大量的应用到了操作系统和一些中间件的底层设计.

比如说, MySQL的buffer pool. buffer pool就是MySQL在内存中的一个缓冲池, 本质就是一个缓存. MySQL插入数据都是先写buffer pool, 然后再某个时间, 异步的刷到磁盘. 然后mysql通过redolog避免数据丢失, 之前讲过了.

再比如说, 操作系统的page cache, 本质也是缓存, 写数据也是写到page cache, 然后操作系统再某个时间, 异步的刷盘.

其实一些业务系统中也有这种思想, 比如现在很多文档编辑器, 或者网上的画图软件, 你不点保存, 它会在某个时间, 自动帮你保存到磁盘, 或者保存到云端. 这也是一个写回的思想.

怎么保证缓存和数据库的一致性

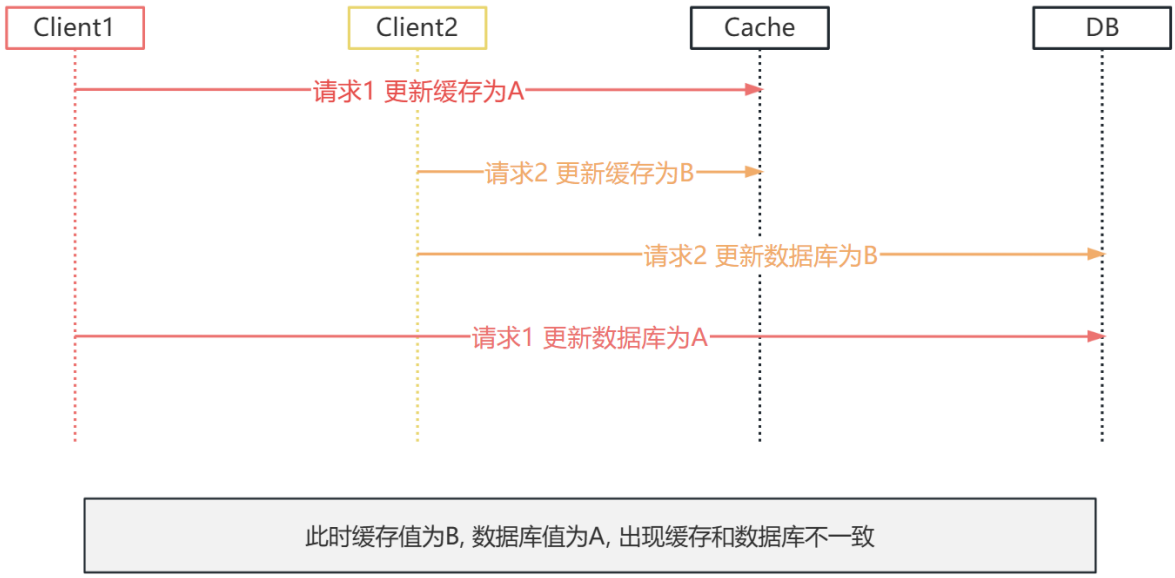
那我们回头看看最常用的旁路缓存, 怎么去读写才能保证缓存和数据库的一致性.

先更新缓存, 再更新数据库

比如说, 我先更新缓存, 再更新数据库.

- 请求1更新缓存为A
- 请求2更新缓存为B
- 请求2更新数据库为B
- 请求1更新数据库为A.

缓存和数据库不一致了. 因为更新缓存和数据库的操作不是原子性的, 中间就会有其他请求来横插一脚, 就会出现并发问题.

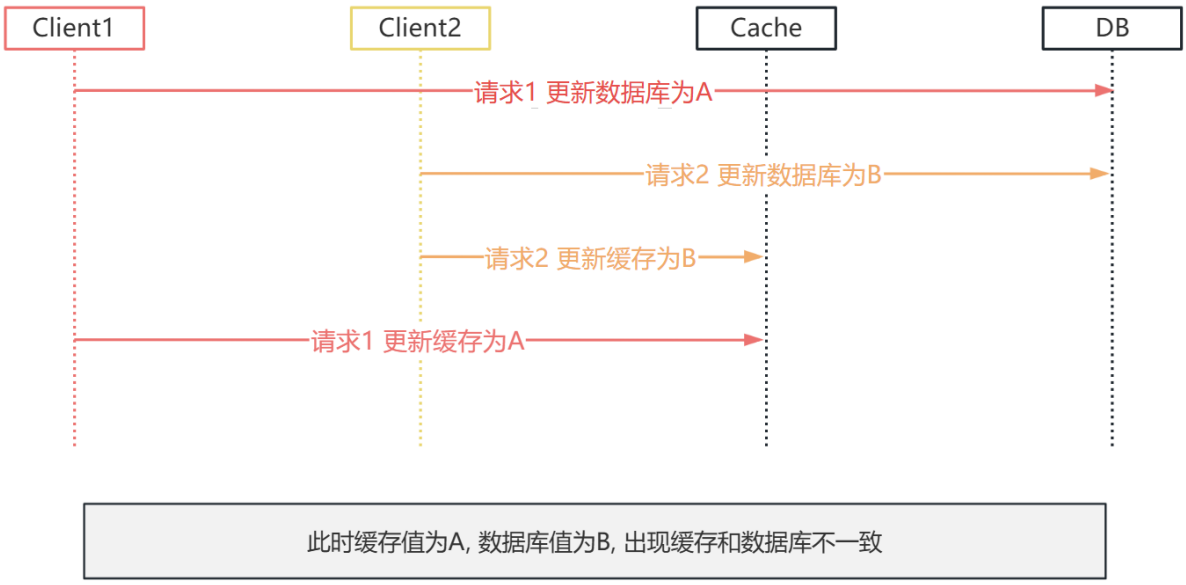


先更新数据库, 然后再更新缓存

换个思路, 先更新数据库, 然后再更新缓存.

- 请求1更新数据库为A
- 请求2更新数据库为B
- 请求2更新缓存为B
- 请求1更新数据库为A.

缓存和数据库又不一致了. 还是那句话, 更新数据库和更新缓存不是原子性的, 并发环境下就会不一致.



换句话说只要能保证这俩操作的原子性, 就能保证强一致. 那你想到了什么?

- 加分布式锁
- 拿到锁的线程, 才能更新数据库, 更新缓存. 这就能保证强一致性
- 但是这样的话, 就不能并发写了, 写操作多了, 就会有性能问题.
- 而且分布式锁自己本身就有很多无法解决的问题, 分布式锁那个文件详细讲过了.

之前有人说过一个分布式读写锁的方案

- 读读不互斥, 所以可以并发读

- 但是写写互斥, 不能并发写, 所以也能保证一致.

看着很美好, 但是这种方案有巨大巨大巨大的缺陷, 千万不能用啊

- 读写锁, 读写互斥. 读的时候是不允许写的, 写的时候也是不允许读的.
- 也就是说高并发情况下, 大量请求都在读这个key, 此时你完全不能改. 或者说当你正在改一个key, 此时大量读请求过来, 是不能读的.
- 所以这种方案问题很大, 还不如缓存数据库直接双写然后加锁的方案

先删除缓存, 再更新数据库

加锁影响性能, 那有没有什么更好的方案呢?

- 比如说我不更新缓存了, 我删除缓存.

那为啥要删除缓存而不是更新缓存?

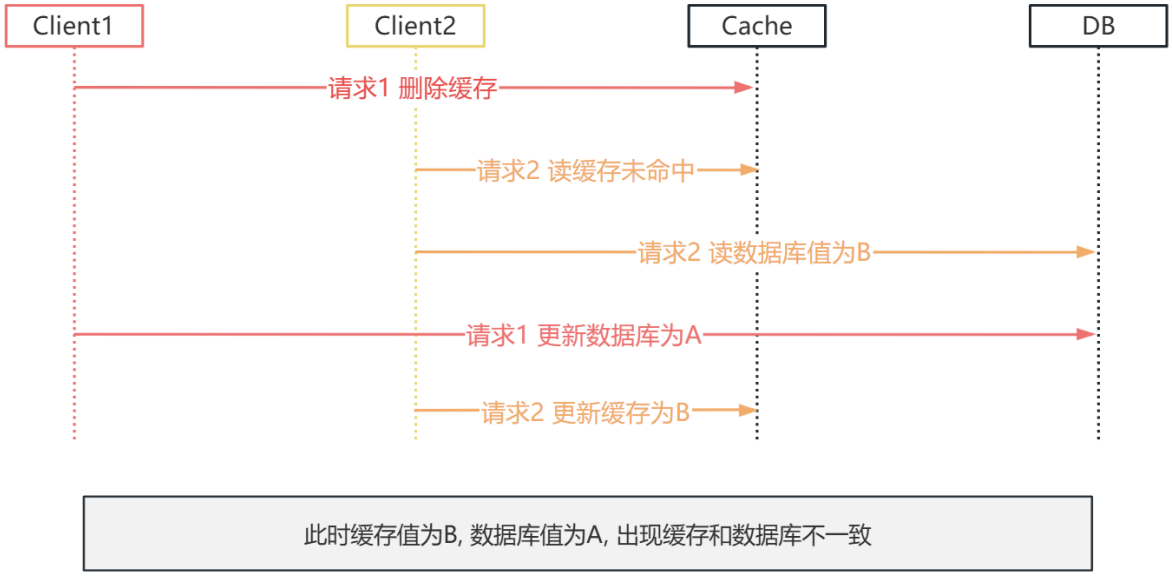
- 因为删除缓存的代价肯定要比更新缓存更低呀
- 缓存的数据基本都要多表联查去聚合然后计算, 那肯定比直接删缓存费劲多了.
- 而且你更新了缓存, 要是一直没被访问, 然后数据库数据又更新了, 你又得更新一遍缓存.
- 所以缓存这个东西, 最好是查的时候, 再写缓存. 更新数据库的时候, 就直接删除缓存.

那就先删除缓存, 再更新数据库.

- 请求1删除缓存
- 请求2读缓存发现没数据, 然后读数据库值为B

- 请求1更新数据库为A
- 请求2写缓存为B.

缓存和数据库又又不一致了.



延迟双删

删一次缓存不好使, 那就等一会儿再删一次缓存, 这叫延迟双删.

- 具体操作就是先删除缓存, 再更新数据库, 线程等待一会儿, 然后再删一次缓存.

为啥等一会儿再删缓存呢?

- 这是要让另一个线程能在等一会儿的这个时间完成读数据库并写缓存的操作, 然后再给他删了.

那么问题来了, 延迟双删, 你要延迟多久, 你要等多久, 到底多久另一个线程才能完成读数据库+写缓存的操作?

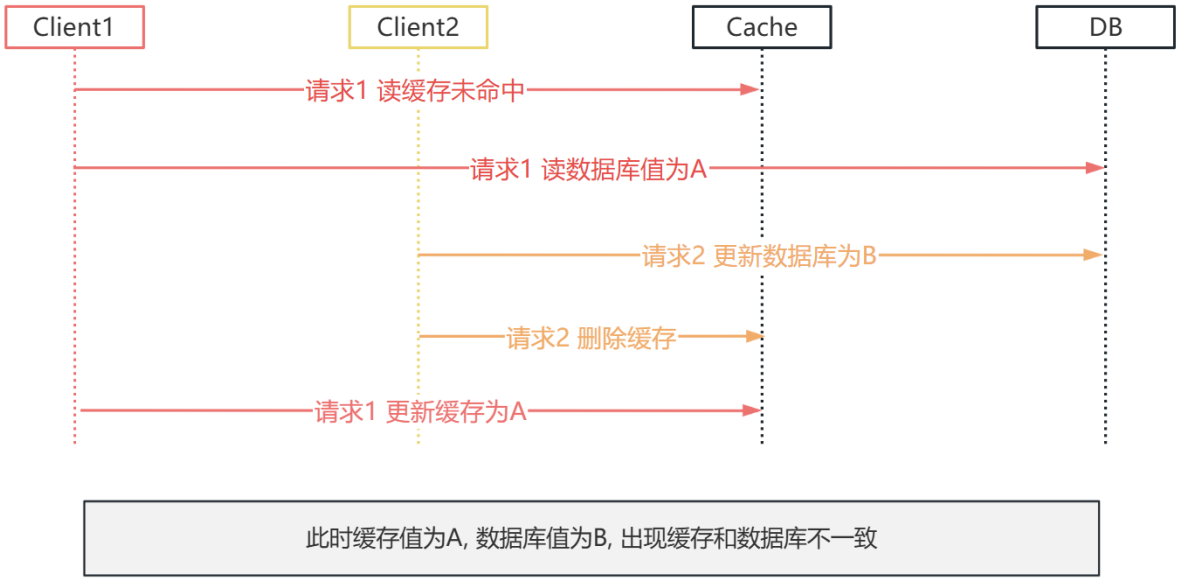
- 这要考虑当时的网络情况, 服务器和数据库的负载等等, 没个准.
- 所以这种方案也是有问题的.

先更新数据库, 再删除缓存

那再再换个思路, 我先更新数据库, 再删除缓存.

- 请求1读缓存, 发现数据过期失效了, 没读到
- 然后请求1就读数据库, 读到值为A.
- 请求2更新数据库为B
- 请求2删除缓存
- 请求1写缓存为A.

缓存和数据库又又又不一致了.



虽然还是有可能不一致, 但是这种概率非常小

- 因为必须得是这个key刚好过期, 然后刚好又要更新数据. 这是第一个条件.
- 第二个条件, 就是请求2更新完数据库, 然后删除缓存之后, 请求1才写入缓存.
- 也就是写缓存的速度要慢于更新数据库+删除缓存的速度. 这个概率是非常低的.
- 所以先更新数据库, 再删除缓存这种操作在绝大多数情况下是可行的.

那么问题来了, 如果更新数据库成功, 删除缓存失败了呢?

- 那缓存和数据库又又又又不一致了.
- 所以虽然这个思路可行, 但是你得保证删除缓存必须成功.

怎么保证删除缓存必须成功?

- 方案1: 把删除缓存的操作放入mq中, mq异步地删除缓存. 如果删除失败, 那就重试.
- 方案2: 监听数据库binlog. 更新数据库成功, 那么就会产生一条binlog, 监听这个binlog, 然后删除缓存, 同样的, 删除失败了就重试. 这也能解决问题, 而且优点是, 完全不入侵业务代码.

所以比较好的方案, 就是先更新数据库, 然后通过mq或者监听binlog的方式异步删缓存, 然后删除失败了就重试.

那么问题又来了, 实际上公司的数据库都是主从部署读写分离的, 主库只能写, 从库只能读, 主库更新了数据, 然后同步给从库.

- 请求1先更新数据库, 然后异步删除了缓存
- 但是你写的是主库, 从库需要等待主库同步数据
- 那从库还没等到主库的数据, 又来一个请求, 读了从库的旧数据, 然后写了缓存

缓存和数据库又又又又又不一致了. 当然了, 这种概率其实也是非常非常低的. 如果要解决这种问题, 那就需要在主库更新后的一段时间内后让其他请求强制读主.

总结

你想完全保证缓存和数据库的一致性, 是几乎不可能的.

我们常用的手段一般都是先更新数据库, 然后通过mq或者监听binlog的方式异步删缓存, 然后删除失败了就重试.

这也只是尽可能保证缓存和数据库在绝大多数情况下是一致的, 然后要设置好数据的过期时间, 即便非常极端的情况下不一致了, 那也能让脏数据过期失效.